

Constraint Satisfaction Problems

Ole C. Eidheim

September 9, 2026

Department of Computer Science

Motivation: the Zebra Puzzle

Five persons of different nationalities and with different jobs live in consecutive houses. The houses are painted in different colors, and the persons have different pets and favorite drinks. Additionally:

- The English lives in a red house
- The Spaniard owns a dog
- The Japanese is a painter
- The Ukrainian drinks tea
- The Norwegian lives in the first house
- The green house immediately to the right of the white one
- The photographer owns snails
- The diplomat lives in the yellow house
- Milk is drunk in the middle house
- The owner of the green house drinks coffee
- The Norwegian's house is next to the blue one
- The violinist drinks orange juice
- The fox is in a house next to that of the physician
- The horse is in a house next to that of the diplomat

Who owns a zebra? And whose favorite drink is water?

Outline

Constraint Satisfaction Problems

Inference: reducing domains

Heuristics: selecting variable and ordering values

Local search

Example CSP software

Search problems vs constraint satisfaction problems

- Search problems
 - Problem specific states, actions, next states, costs, end state, search function, and heuristics
- Constraint satisfaction problems
 - A set of constraints that must be satisfied
 - Problem specific variables, variable domains, and constraints
 - Problem independent search function, inference and heuristics
 - Example problem: Sudoku
 - Variables: the cells
 - Variable domains: $\{1, \dots, 9\}$
 - Constraints: different values horizontally, vertically, and within 3x3 blocks

		3		2		6		
9			3		5			1
		1	8		6	4		
		8	1		2	9		
7								8
		6	7		8	2		
		2	6		9	5		
8			2		3			9
		5		1		3		

Constraint Satisfaction Problems

Definitions

- Variables $X_1, \dots, X_n$
 - The variables might have different names, e.g. $A, B, \dots$
- Domains $D_1, \dots, D_n$ of each variable $X_i \in D_i$, i.e. values each variable can have
- Constraints $C_1, \dots, C_m$ with each $C_j(\dots) \in \{true, false\}$

For example: $C_1(X_1, X_2) = \begin{cases} true, & \text{if } X_1 \neq X_2 \\ false, & \text{otherwise} \end{cases}$

or in book notation: $C_1 : \langle (X_1, X_2), X_1 \neq X_2 \rangle$

or simply $C_1 : X_1 \neq X_2$

Objective: find a complete assignment $\{X_1 = v_1, \dots, X_n = v_n\}$ that satisfies all constraints

Example: coloring problem

Assign red, green or blue to each region, but neighboring regions cannot have the same color

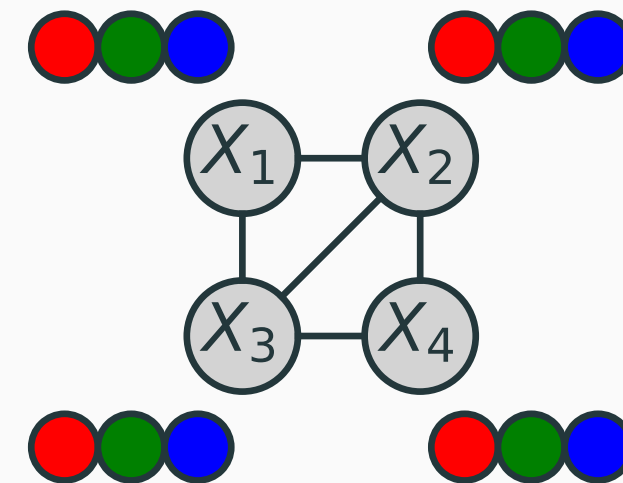


Example: coloring problem

Assign red, green or blue to each region, but neighboring regions cannot have the same color, i.e. constraints: $\{X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3, X_2 \neq X_4, X_3 \neq X_4\}$



For simplicity, ignoring NSW, V, T



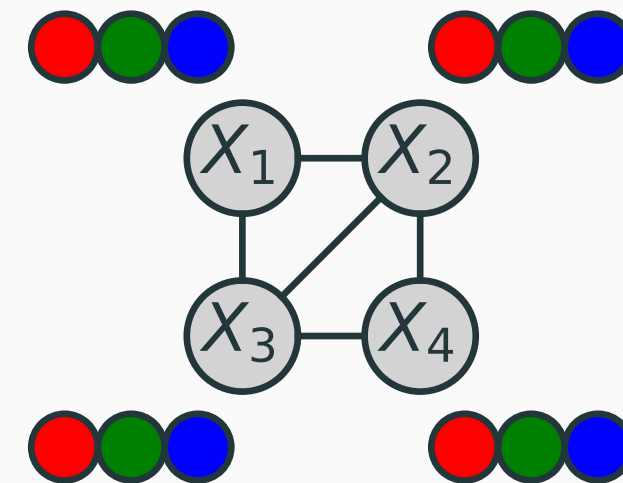
Constraint graph with domains

Example: coloring problem

Assign red, green or blue to each region, but neighboring regions cannot have the same color, i.e. constraints: $\{X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3, X_2 \neq X_4, X_3 \neq X_4\}$



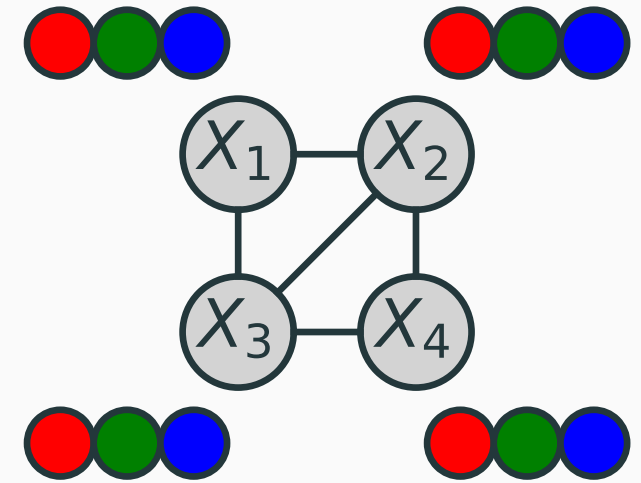
For simplicity, ignoring NSW, V, T



Constraint graph with domains

Example code solution (CSP implementation not shown)

More on constraints: coloring problem



- Variables: X_1, X_2, X_3, X_4
- Domains: $D_1 = \dots = D_4 = \{red, green, blue\}$
- Constraints: $\{X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3, X_2 \neq X_4, X_3 \neq X_4\}$

or as tuples of values:

$\{\langle (X_1, X_2), \{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\} \rangle, \langle (X_1, X_3), \{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\} \rangle, \langle (X_2, X_3), \{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\} \rangle, \langle (X_2, X_4), \{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\} \rangle, \langle (X_3, X_4), \{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\} \rangle\}$

More on constraints: a simpler problem

- Variables: X_1, X_2
- Domains: $D_1 = \{1, 2, 3, 4, 5\}, D_2 = \{1, 2\}$
- Constraints: $\{X_1 + X_2 = 4\}$
or as tuples of values:
 $\{\langle (X_1, X_2), \{(2, 2), (3, 1)\} \rangle\}$

- $alldiff(variables)$: inequality constraints between the *variables*
 - For example: $alldiff(X_1, X_2, X_3) = \{X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3\}$

Backtracking search for Constraint Satisfaction Problems

Algorithm

```
function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences  $\leftarrow$  INFERENCE(csp, var, assignment)
      if inferences  $\neq$  failure then
        add inferences to csp
        result  $\leftarrow$  BACKTRACK(csp, assignment)
        if result  $\neq$  failure then return result
        remove inferences from csp
      remove {var = value} from assignment
  return failure
```

Outline

Constraint Satisfaction Problems

Inference: reducing domains

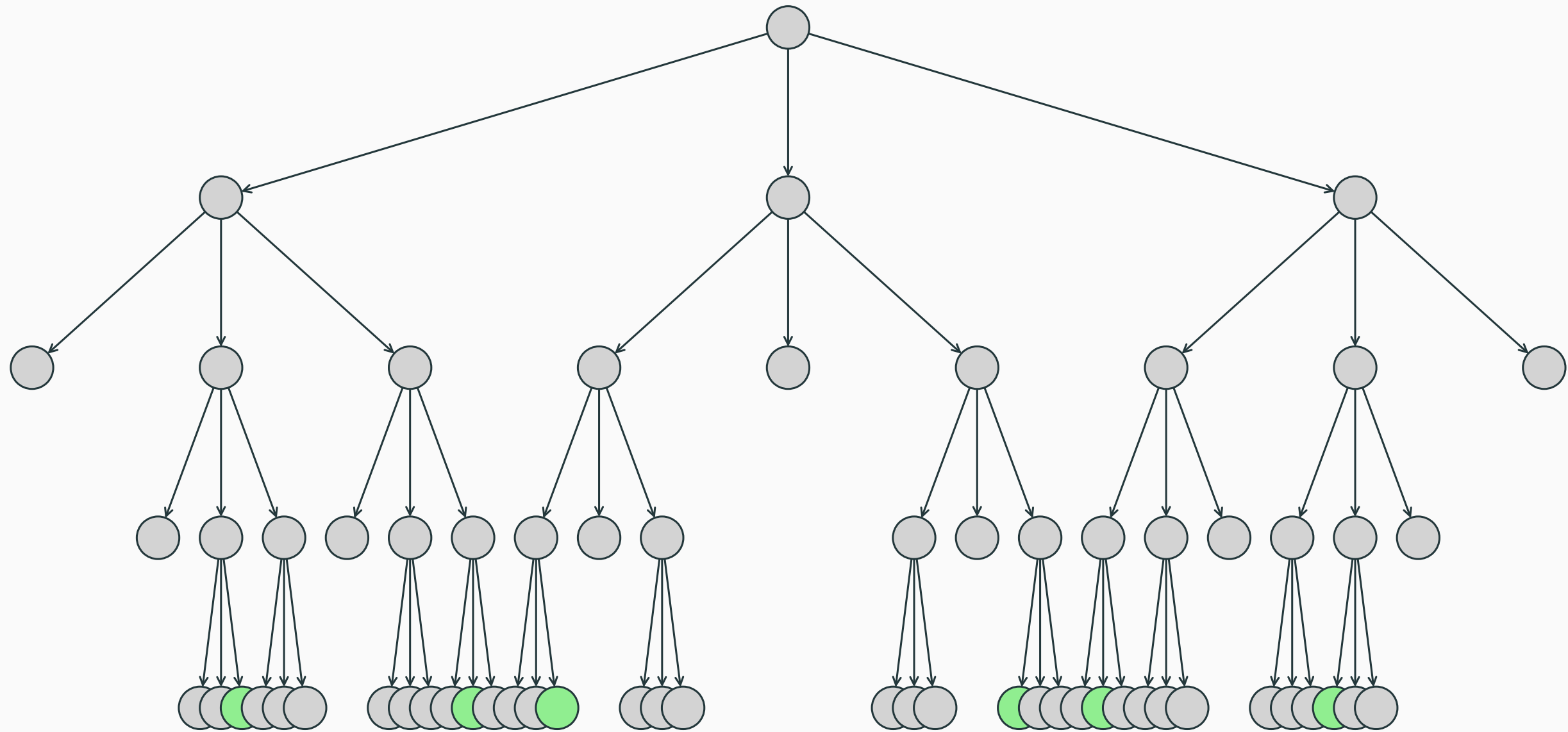
Heuristics: selecting variable and ordering values

Local search

Example CSP software

Motivation: simplify search by solving subnetworks

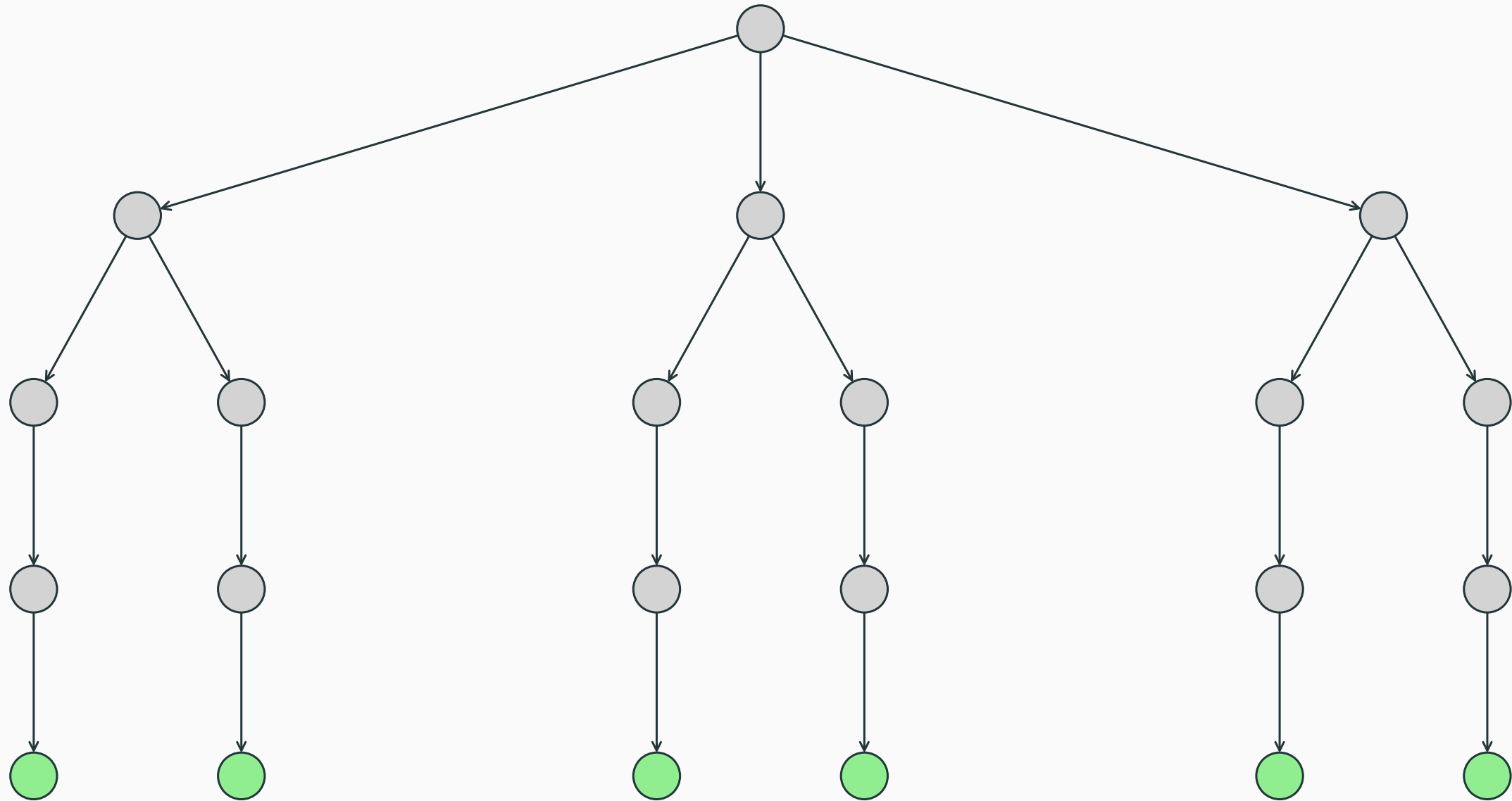
Without inference:



Example search tree with all solutions shown

Motivation: simplify search by solving subnetworks

With inference:



Example search tree with all solutions shown

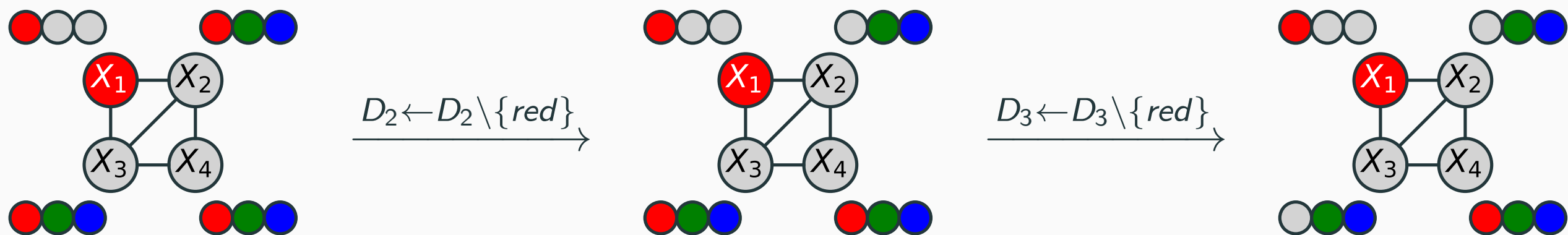
Forward checking

Algorithm

After assigning a value to a variable, explore the domains of neighboring unassigned variables, and remove values whose assignments would violate a constraint

- Neighboring variables: variables that share constraint dependencies
- During inference, domains are also set to the assigned value

For example, after assigning $\{X_1 = red\}$ (and $D_1 \leftarrow \{red\}$):



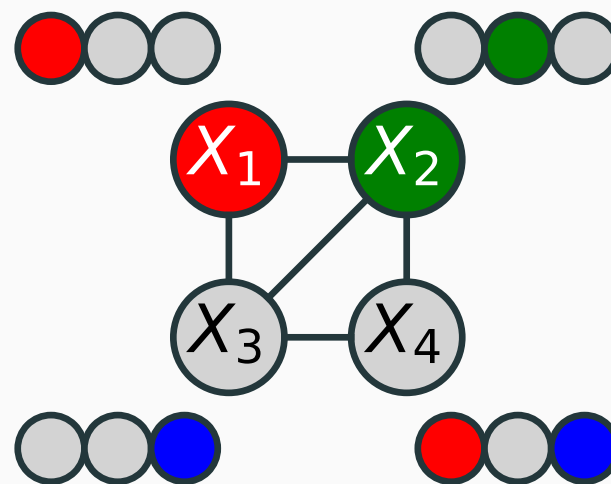
Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

For example, is (X_3, X_4) arc-consistent (is X_3 arc-consistent with X_4)?



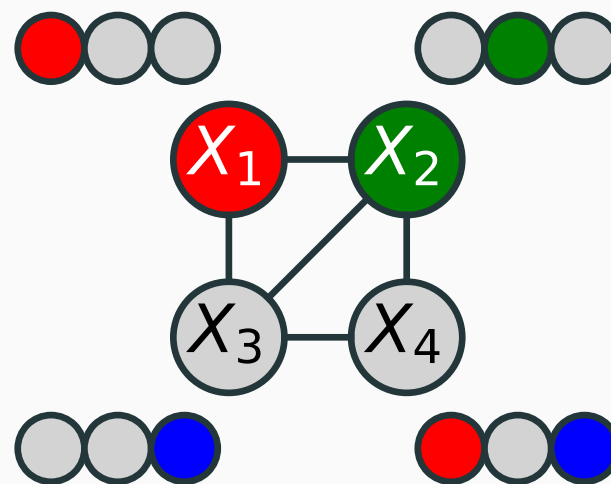
Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

For example, is (X_3, X_4) arc-consistent (is X_3 arc-consistent with X_4)?



Answer: yes

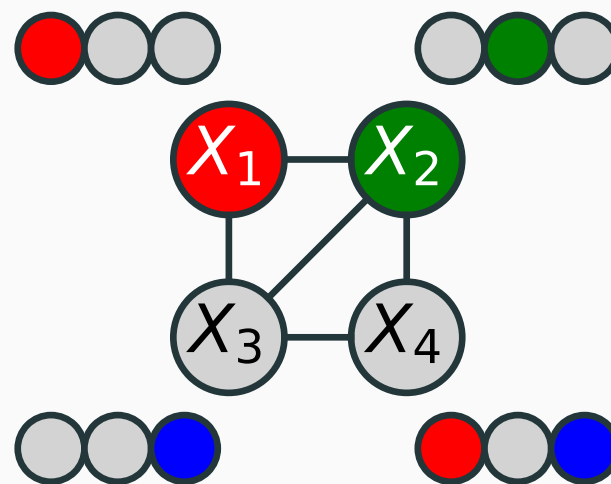
Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

For example, is (X_4, X_3) arc-consistent?



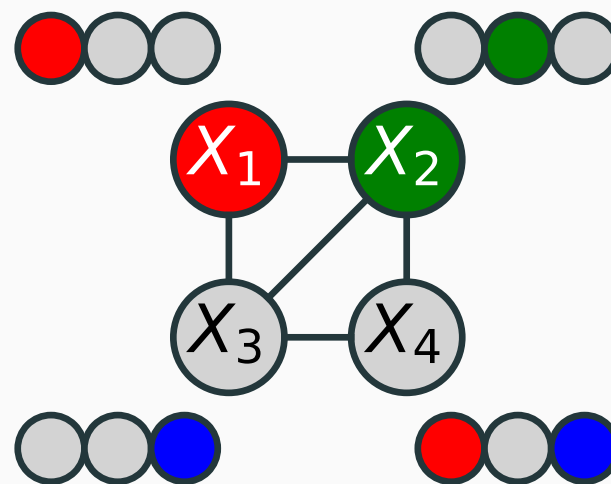
Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

For example, is (X_4, X_3) arc-consistent?



Answer: no

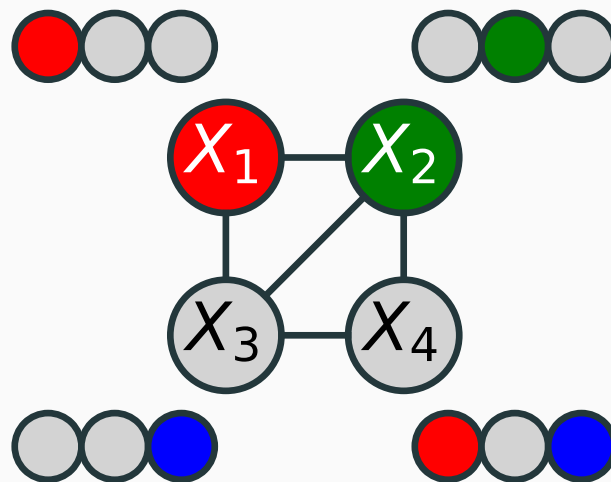
Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

How do we make (X_4, X_3) arc-consistent?



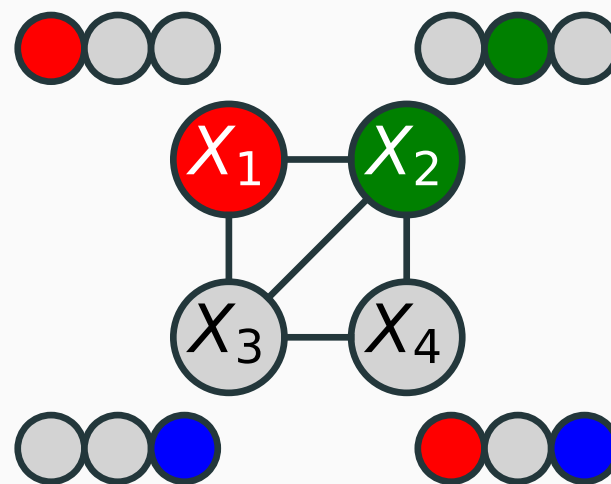
Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

How do we make (X_4, X_3) arc-consistent?



Answer: $D_4 \leftarrow D_4 \setminus \{blue\}$

Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

Make (X_i, X_j) arc-consistent

Remove the values from D_i that make (X_i, X_j) arc-inconsistent

Arc Consistency Algorithm #3 (AC-3)

Arc-consistency

X_i is arc-consistent with X_j if for every value in D_i there exists a value in D_j whose assignments would not violate any constraint

- Only binary (two-variable) constraints considered

Make (X_i, X_j) arc-consistent

Remove the values from D_i that make (X_i, X_j) arc-inconsistent

AC-3 Algorithm

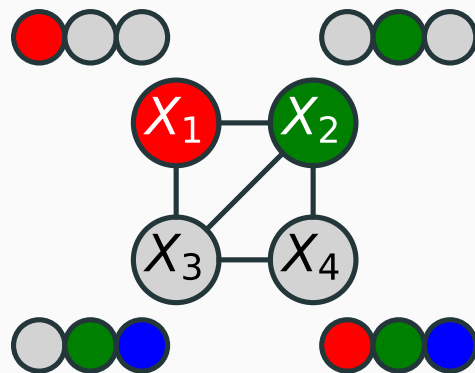
- start with an initial queue of arcs
- while queue is not empty
 - pop an arc and make arc-consistent
 - if the domain was reduced for a variable, add arcs from its unassigned neighbors to the variable

Arc Consistency Algorithm #3 (AC-3)

One way is to run AC-3 after assigning a variable, with arcs from its neighboring unassigned variables to the variable in the initial queue

For example, after assigning $\{X_2 = \text{green}\}$ (and $D_2 \leftarrow \{\text{green}\}$):

- Starts with two arcs in the queue: (X_3, X_2) and (X_4, X_2)

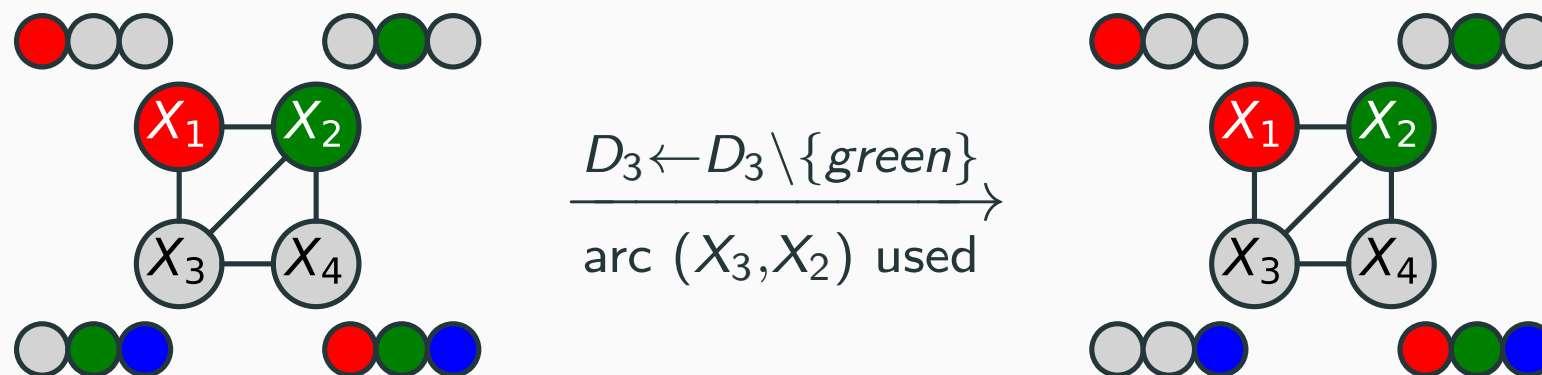


Arc Consistency Algorithm #3 (AC-3)

One way is to run AC-3 after assigning a variable, with arcs from its neighboring unassigned variables to the variable in the initial queue

For example, after assigning $\{X_2 = \text{green}\}$ (and $D_2 \leftarrow \{\text{green}\}$):

- Starts with two arcs in the queue: (X_3, X_2) and (X_4, X_2)
- When D_3 is updated, the arc (X_4, X_3) is added to queue

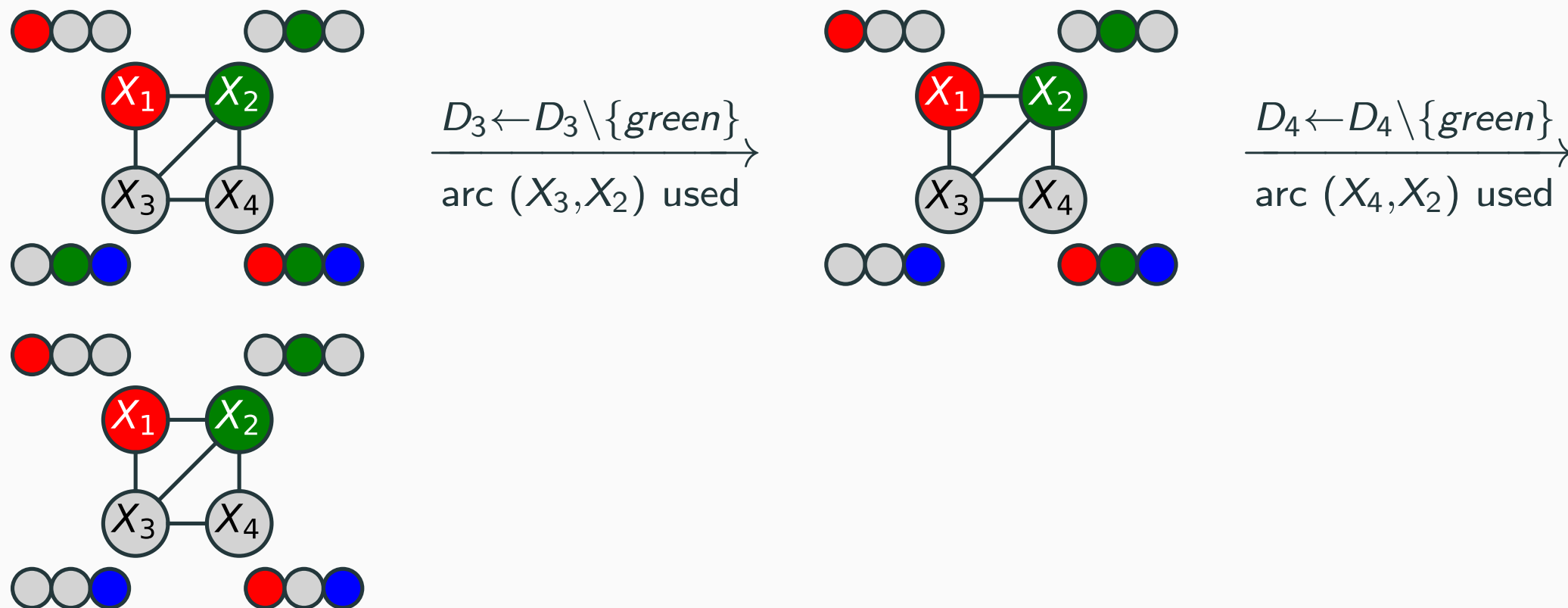


Arc Consistency Algorithm #3 (AC-3)

One way is to run AC-3 after assigning a variable, with arcs from its neighboring unassigned variables to the variable in the initial queue

For example, after assigning $\{X_2 = green\}$ (and $D_2 \leftarrow \{green\}$):

- Starts with two arcs in the queue: (X_3, X_2) and (X_4, X_2)
- When D_3 is updated, the arc (X_4, X_3) is added to queue
- When D_4 is updated, the arc (X_3, X_4) is added to queue

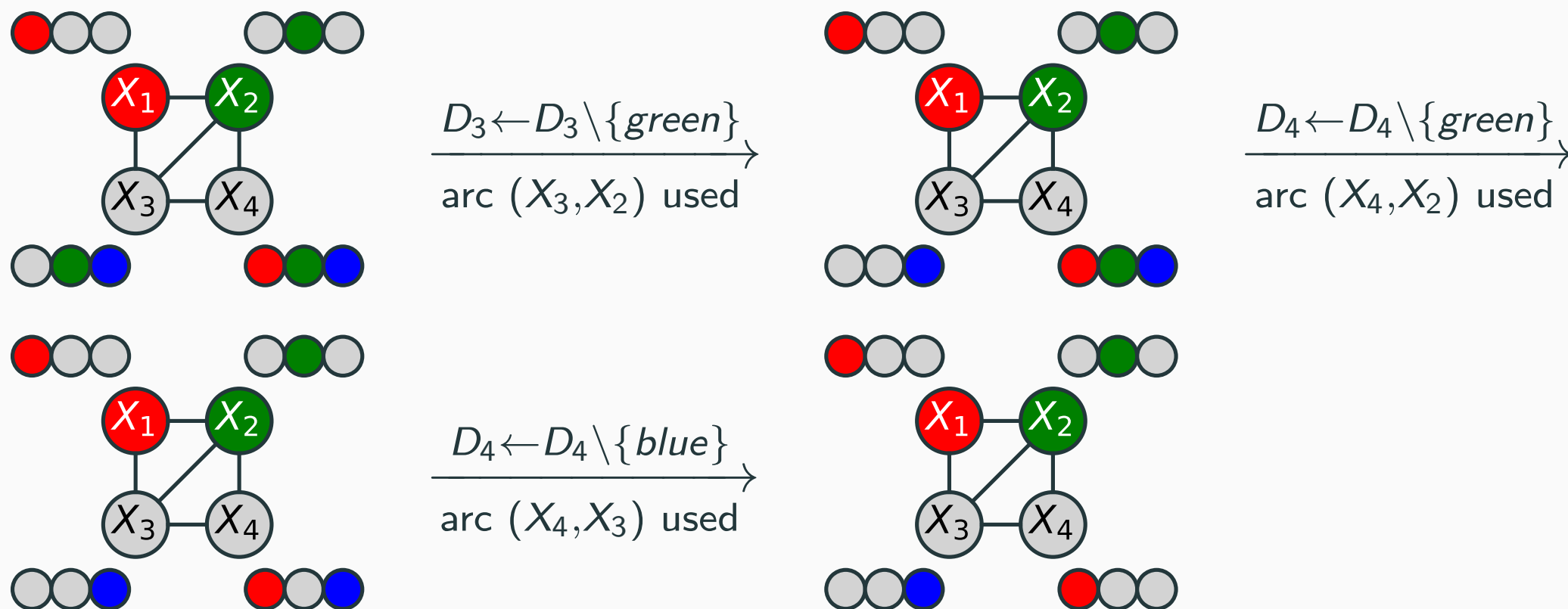


Arc Consistency Algorithm #3 (AC-3)

One way is to run AC-3 after assigning a variable, with arcs from its neighboring unassigned variables to the variable in the initial queue

For example, after assigning $\{X_2 = green\}$ (and $D_2 \leftarrow \{green\}$):

- Starts with two arcs in the queue: (X_3, X_2) and (X_4, X_2)
- When D_3 is updated, the arc (X_4, X_3) is added to queue
- When D_4 is updated, the arc (X_3, X_4) is added to queue



Arc Consistency Algorithm #3 (AC-3)

Another way is to execute AC-3 only once, but before backtracking search, with arcs to and from all neighboring unassigned variables in the initial queue (see the algorithm in Figure 5.3 in the book)

This can for example solve simple Sudoku problems without search:

	4		1
3			
			4

Domains after setting the initial values:

$$D_{12} = \{4\}, D_{14} = \{1\}, D_{21} = \{3\}, D_{34} = \{4\},$$

$$D_{11} = D_{13} = D_{22} = \dots = D_{33} = D_{41} = \dots = D_{44} = \{1, 2, 3, 4\}$$

Arc Consistency Algorithm #3 (AC-3)

Another way is to execute AC-3 only once, but before backtracking search, with arcs to and from all neighboring unassigned variables in the initial queue (see the algorithm in Figure 5.3 in the book)

This can for example solve simple Sudoku problems without search:

	4		1
3			
			4

Domains after setting the initial values:

$$D_{12} = \{4\}, D_{14} = \{1\}, D_{21} = \{3\}, D_{34} = \{4\},$$

$$D_{11} = D_{13} = D_{22} = \dots = D_{33} = D_{41} = \dots = D_{44} = \{1, 2, 3, 4\}$$

Domains after running the AC-3 algorithm:

$$D_{11} = \{2\}, D_{12} = \{4\}, D_{13} = \{3\}, D_{14} = \{1\}, D_{21} = \{3\}, D_{22} = \{1\}, D_{23} = \{4\}, D_{24} = \{2\},$$

$$D_{31} = \{1\}, D_{32} = \{3\}, D_{33} = \{2\}, D_{34} = \{4\}, D_{41} = \{4\}, D_{42} = \{2\}, D_{43} = \{1\}, D_{44} = \{3\}$$

Outline

Constraint Satisfaction Problems

Inference: reducing domains

Heuristics: selecting variable and ordering values

Local search

Example CSP software

Possibly faster search by focusing on likely good choices

Backtracking search for Constraint Satisfaction Problems

Algorithm

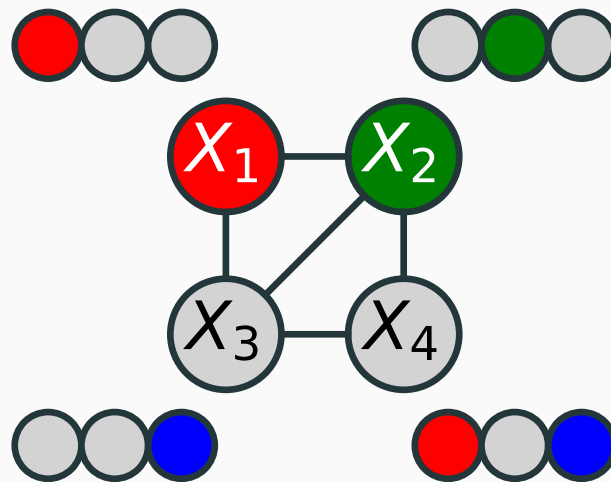
```
function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences  $\leftarrow$  INFERENCE(csp, var, assignment)
      if inferences  $\neq$  failure then
        add inferences to csp
        result  $\leftarrow$  BACKTRACK(csp, assignment)
        if result  $\neq$  failure then return result
        remove inferences from csp
      remove {var = value} from assignment
  return failure
```

Selecting variable

Choose the most constrained variable

Select the unassigned variable with the smallest remaining domain

For example, will X_3 or X_4 be chosen?

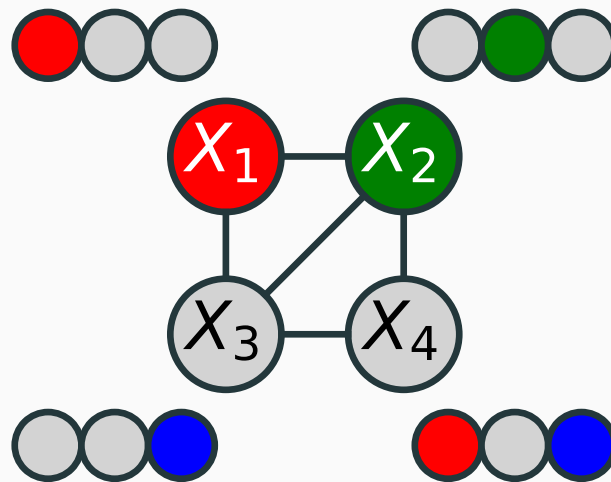


Selecting variable

Choose the most constrained variable

Select the unassigned variable with the smallest remaining domain

For example, will X_3 or X_4 be chosen?



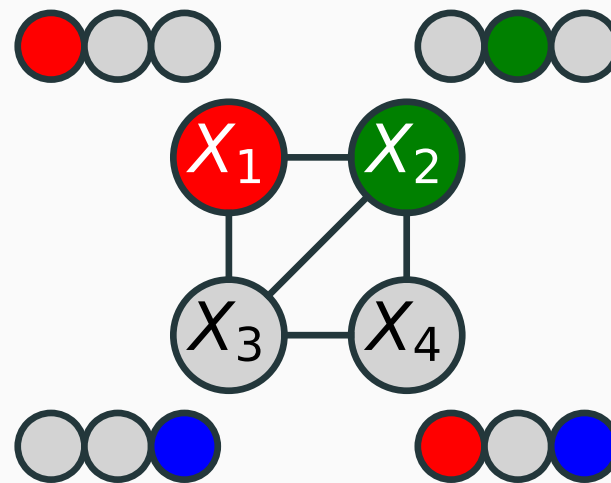
Answer: X_3

Ordering values

Choose the least constrained values first

Prefer values that would cause the smallest reduction in the unassigned variable domains during forward checking

For example, given variable X_4 was chosen, will red or blue be chosen?

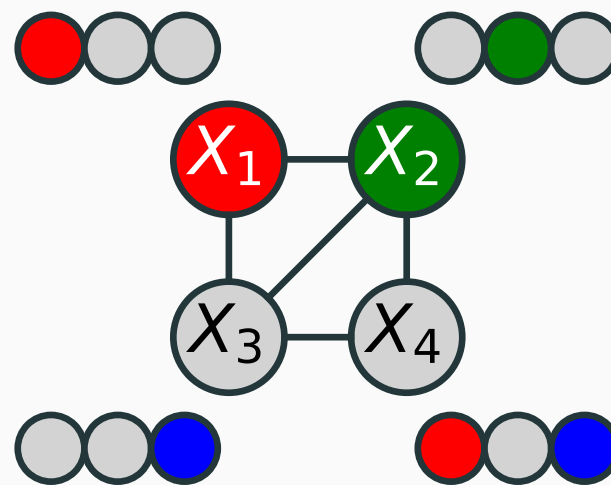


Ordering values

Choose the least constrained values first

Prefer values that would cause the smallest reduction in the unassigned variable domains during forward checking

For example, given variable X_4 was chosen, will red or blue be chosen?



Answer: red

Example: cryptarithmic problem

Assign distinct digits to each letter such that the sum is correct, and none of the numbers have leading zeros, i.e. $T \neq 0$ and $F \neq 0$

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$

Example solution:

$$\begin{array}{r} 7 \ 6 \ 5 \\ + \ 7 \ 6 \ 5 \\ \hline 1 \ 5 \ 3 \ 0 \end{array}$$

Outline

Constraint Satisfaction Problems

Inference: reducing domains

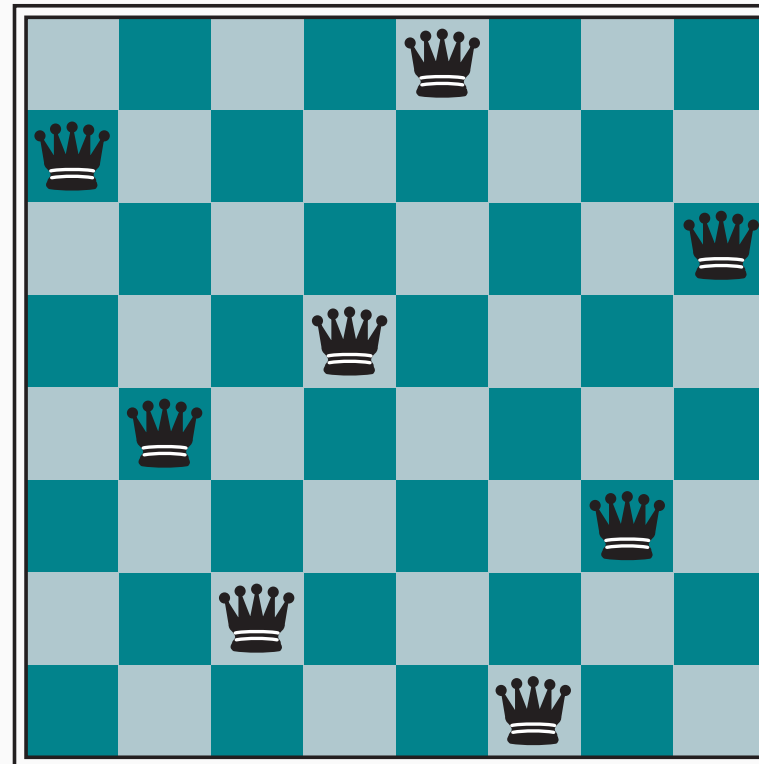
Heuristics: selecting variable and ordering values

Local search

Example CSP software

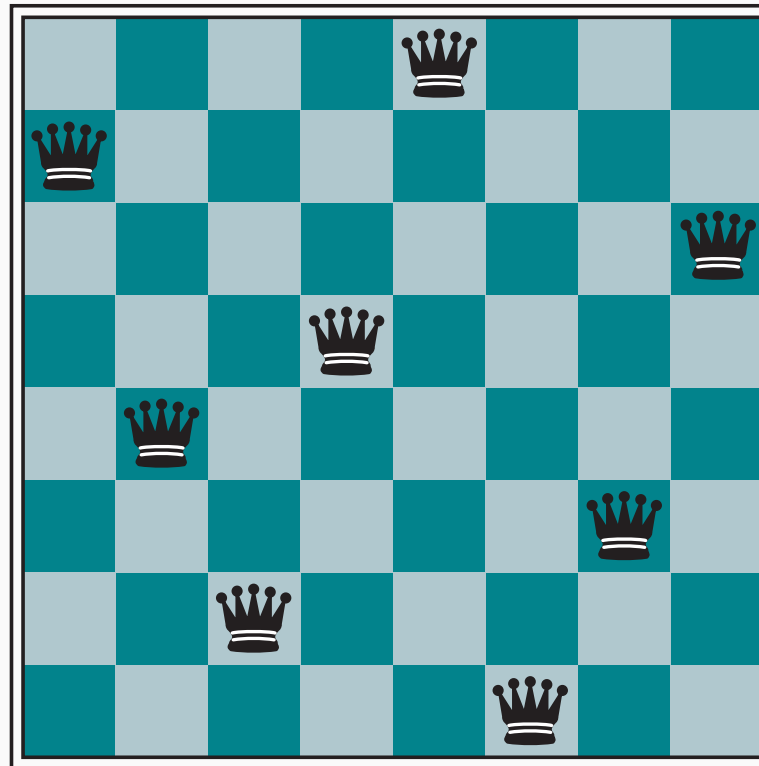
Example: 8-queens problem

Example solution:



Example: 8-queens problem

Example solution:



What can the CSP variables and domains be?

Min-Conflicts

function MIN-CONFLICTS(csp , max_steps) **returns** a solution or *failure*

inputs: csp , a constraint satisfaction problem

max_steps , the number of steps allowed before giving up

$current \leftarrow$ an initial complete assignment for csp

for $i = 1$ to max_steps **do**

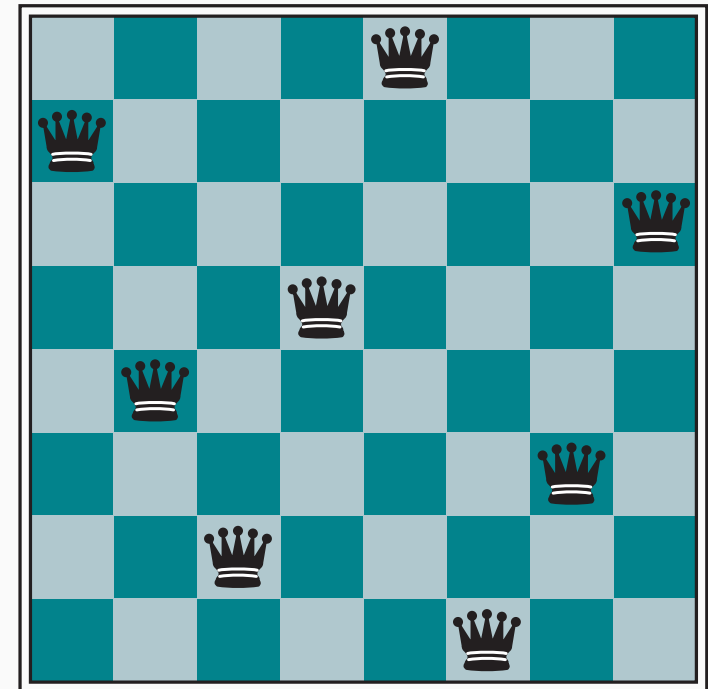
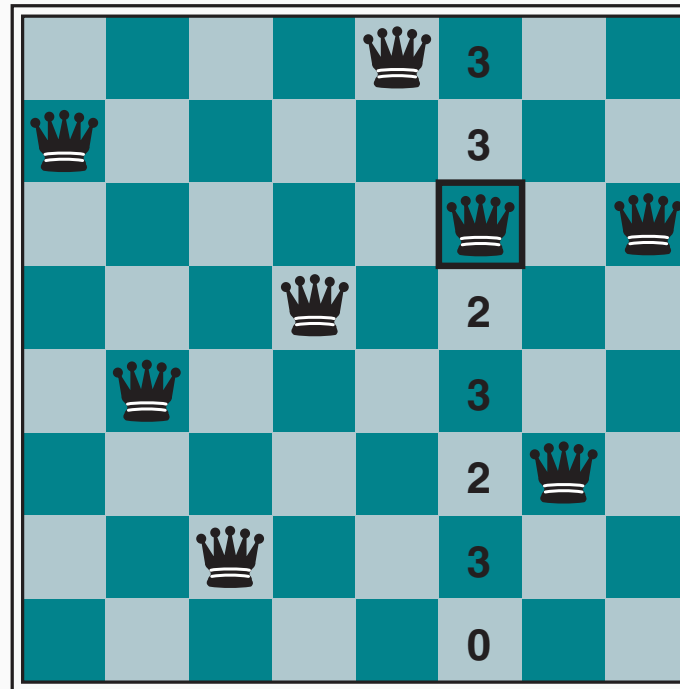
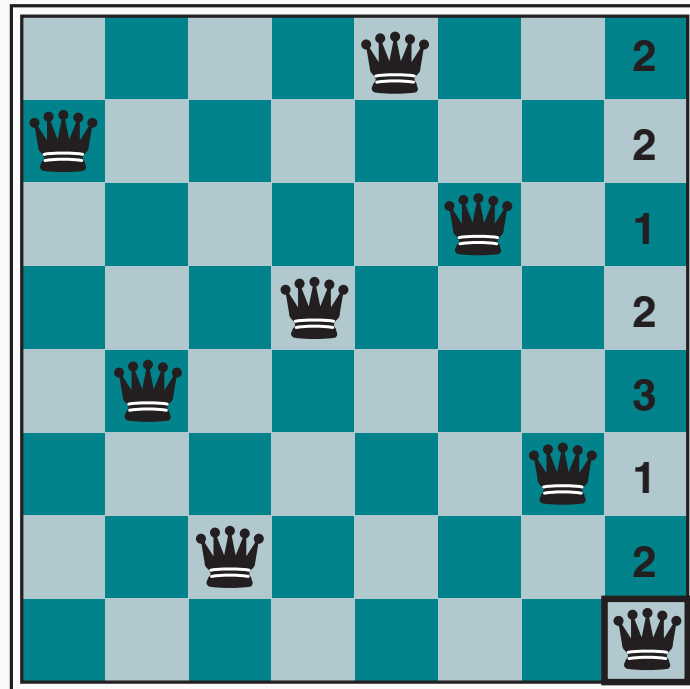
if $current$ is a solution for csp **then return** $current$

$var \leftarrow$ a randomly chosen conflicted variable from $csp.VARIABLES$

$value \leftarrow$ the value v for var that minimizes CONFLICTS(csp , var , v , $current$)

 set $var = value$ in $current$

return *failure*



Min-Conflicts

function MIN-CONFLICTS(csp , max_steps) **returns** a solution or *failure*

inputs: csp , a constraint satisfaction problem

max_steps , the number of steps allowed before giving up

$current \leftarrow$ an initial complete assignment for csp

for $i = 1$ to max_steps **do**

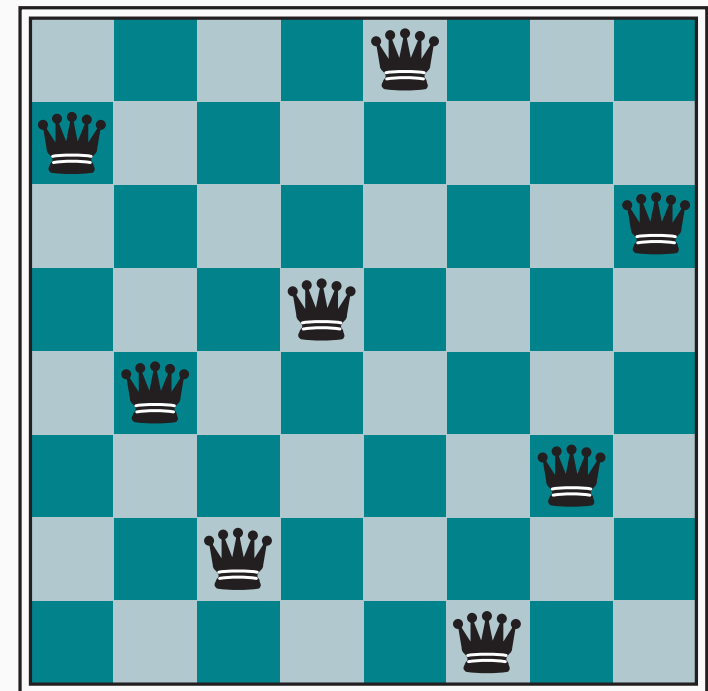
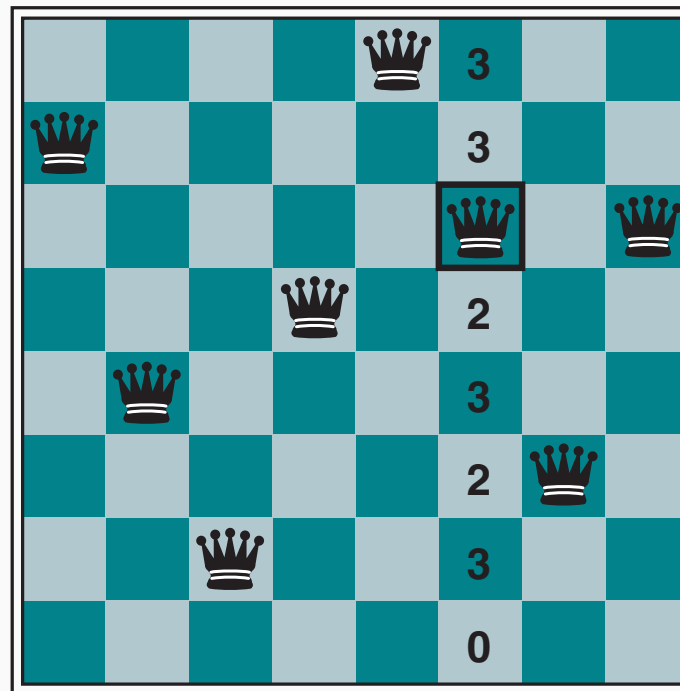
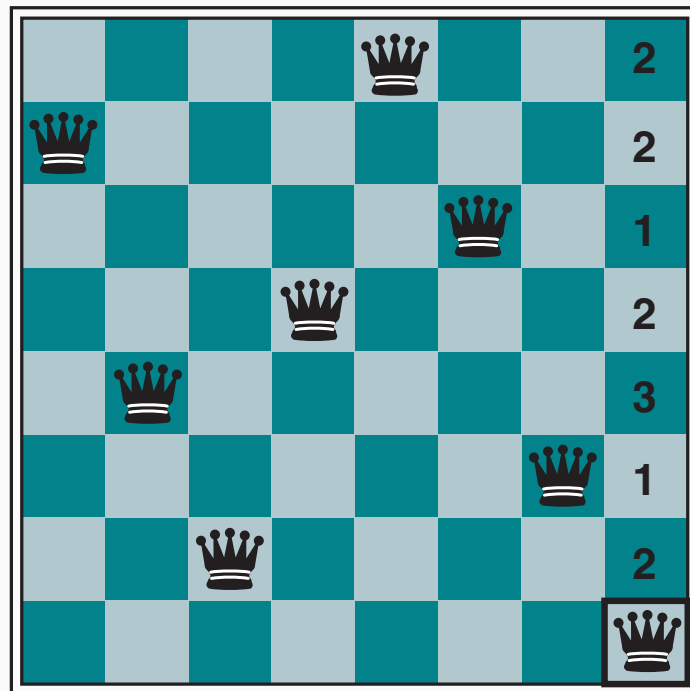
if $current$ is a solution for csp **then return** $current$

$var \leftarrow$ a randomly chosen conflicted variable from $csp.VARIABLES$

$value \leftarrow$ the value v for var that minimizes CONFLICTS(csp , var , v , $current$)

 set $var = value$ in $current$

return *failure*



The MIN-CONFLICTS local search algorithm can solve a million-queens problem in an average of 50 steps

Outline

Constraint Satisfaction Problems

Inference: reducing domains

Heuristics: selecting variable and ordering values

Local search

Example CSP software

Example software that can be used to solve CSPs (not curriculum)

```
from z3 import *

ColorSort, _ = EnumSort('Color', ['red', 'green', 'blue']) # Sort means type
X1, X2, X3, X4 = Consts("X1 X2 X3 X4", ColorSort)

s = Solver()

s.add(X1 != X2)
s.add(X1 != X3)
s.add(X2 != X3)
s.add(X2 != X4)
s.add(X3 != X4)

print(s.check())
print(s.model())

# Example output:
# sat
# [X1 = red, X2 = green, X3 = blue, X4 = red]
```